Aula 3 – Pilhas

Classes genéricas em Java

Pilhas

Representação de Pilhas com Arrays

Exemplos de Algoritmos usando Pilhas

Outras Representações de Pilhas

Classes Genéricas

Limitações do java até à versão 1.5

```
ArrayList a = new ArrayList();
a.add(new String(“ola”));
a.add(new String(“mundo”));
a.add(new String(“cruel”));

// mostra o tamanho de cada Elemento
for(int i = 0; i < a.size(); i++) {
  sout(“Tamanho:” + a.get(i).length());
}
```

Não podemos aceder ao método length(), porque o get() devolve um Object e não uma String

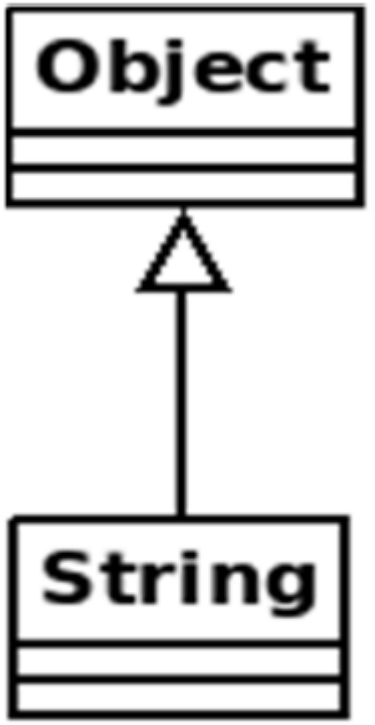
Classes Genéricas

Soluções:

```
for(int i = 0; i < a.getSize(); i++) {
  sout("Tamanho:" + ((String)a.get(i)).length());
}
```

Ou

```
for(int i = 0; i < a.getSize(); i++) {
  String s = (String)a.get(i);
  sout("Tamanho:" + s.length());
}
```



Classes Genéricas

Ou ainda (a partir do Java 1.5):

```
ArrayList<String> a = new ArrayList();
a.add(new String("ola"));
a.add(new String("mundo"));
a.add(new String("cruel"));

// mostra o tamanho de cada Elemento
for(int i = 0; i < a.size(); i++) {
  sout("Tamanho:" + a.get(i).length());
}
```

Ou

```
for(String str : a) {
  sout("Tamanho:" + str.length());
}
```

Classes Genéricas

Classe não genérica:

```
class UmObjecto {
  private Object o;
  public UmObjecto(Object o) {
    this.o = o;
  }
  public Object get() {
   return this.o;
  }
  public void set(Object o) {
   this.o = o;
  }
}
```

Classe genérica:

```
class UmObjecto<T> {
  private T o;
  public UmObjecto(T o) {
    this.o = o;
  }
  public T get() {
   return this.o;
  }
  public void set(T o) {
   this.o = o;
  }
}
```

Classes Genéricas

Classe não genérica:

```
class DoisObjecto {
  private Object o1;
  private Object o2;
  public DoisObjecto(Object o1,
                     Object o2){
    this.o1 = o1;
    this.o2 = o2;
  }
  public Object get1() {
   return this.o1;
  }
  public Object get2() {
   return this.o2;
  }
  public void set1(Object o) {
   this.o1 = o;
  }
  public void set2(Object o) {
   this.o2 = 0;
  }
}
```

Classe genérica:

```
class DoisObjecto<T,U> {
  private T o1;
  private U o2;
  public DoisObjecto(T o1,
                     U o2){
    this.o1 = o1;
    this.o2 = o2;
  }
  public T get1() {
   return this.o1;
  }
  public U get2() {
   return this.o2;
  }
  public void set1(T o) {
   this.o1 = o;
  }
  public void set2(U o) {
   this.o2 = 0;
  }
}
```

Pilhas

- Complexidade Temporal:

Introdução - O(1) ; Eliminação - O(1) ; Pesquisa - O(1)



Introdução: Colocar em cima

Eliminar: Tirar de Cima

Pesquisa: Ver o que está em cima

Pilhas

- Onde se usam:
  - Como estruturas auxiliares na resolução de pequenos problemas
  - Tem um tempo de vida curto:
    - Criadas, Utilizadas, Destruídas
- Limitações:
  - Restrições no acesso

Pilhas

- Métodos usais numa classe que represente uma pilha:
  - boolean empty() - Indica se a Pilha está vazia
  - Object peek() - Devolve o elemento do topo da Pilha sem o remover
  - Object pop() - Remove e devolve o elemento do topo da Pilha
  - void push(Object data) – Coloca um elemento no topo da Pilha

Representação de Pilhas com Arrays

Situação Inicial:

topo = -1



push(4)

topo = 0



push(5)

topo = 1



Representação de Pilhas com Arrays

push(1), push(3), push(8), push(4), push(9)

topo = 6



a = pop() // a = 9

topo = 5



a = pop() // a = 4

topo = 4



Representação de Pilhas com Arrays

a = peek() // a = 8

topo = 4



push(1), push(3), push(8), push(4), push(9)

topo = 9





push(2)

Erro

topo = 9

Empty

```
boolean empty() {
    Se topo = -1
        devolve verdadeiro
    Senão
        devolve falso
}
```

Push & Pop

```
void push(Object data) {
    topo = topo + 1
    array[topo] = data
}
```

```
Object pop() {
    Se empty()
        devolve null
    Senão
        devolve array[topo--]
}
```

Peek

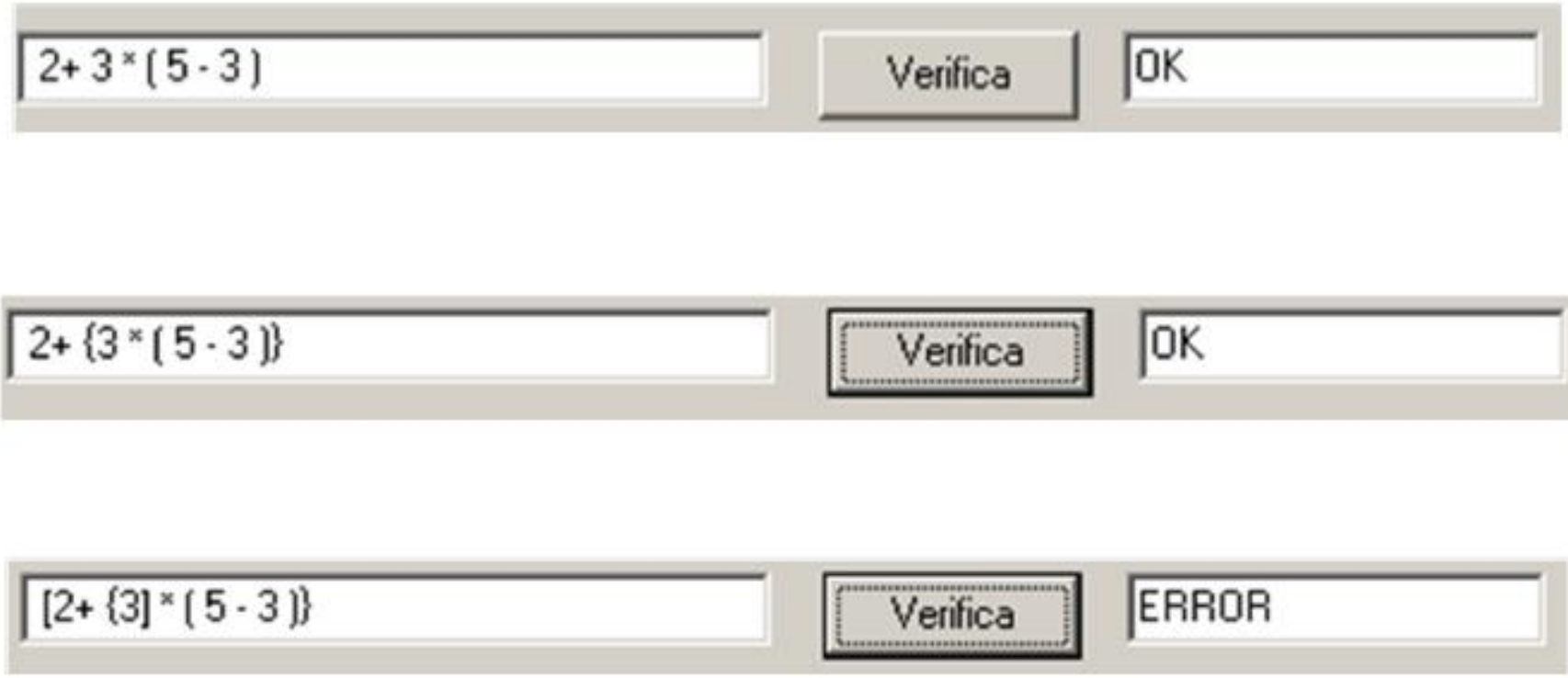
```
Object peek() {
    Se empty()
        devolve null
    Senão
        devolve array[topo]
}
```

Exemplos – Inversão de String




```
Percorrer todos os caracteres da string
    introduzir os caracteres numa pilha
Iniciar uma string str a nulo
Enquanto a pilha não estiver vazia
    retirar o elemento da pilha e
    acrescentar o caracter da pilha a str
```

Exemplos – Verificação de ( ), [ ], { }



Exemplos – Verificação de ( ), [ ], { }

- Para todos os caracteres ch da STRING
  - Se ch é um parêntesis a abrir (**{ ([**)
    - Introduzir ch na pilha
  - Se ch é um parêntesis a fechar (**})]**)
    - Retirar o último elemento chp da pilha
    - Se não fizerem par ch e chp
      - Retorna falso
- Se a pilha contiver elementos
  - Retorna falso
- Senão
  - Retorna verdadeiro

Exemplos – Avaliação de Expressões

- Avaliação Pós-Fixa:
  - 2 + 3 * 5
    - 2 3 5 * +
      - 17
  - 6 * (4 – 2 + 1) / 3
    - 6 4 2 – 1 + * 3 /
      - 6
  - 2 > 3 ou 5 < 4
    - 2 3 > 5 4 < ou
      - falso

Exemplos – Avaliação de Expressões

Algortimo para avaliar uma expressão Pós-Fixa

- Para todos os elementos elem da Expressão
  - Se elem é um operador
    - Retirar X1 da pilha
    - Retirar X2 da pilha
    - Calcular x3 como o resultado de x2 elem x1
    - Introduzir x3 na pilha
  - Senão
    - Introduz na elem pilha
- Retorna o elemento que ficou na pilha;

Exemplos – Avaliação de Expressões

- Avaliação Pós-Fixa com Operadores com vários argumentos:
  - não (2 > 3 ou 5 < 4)
    - 2 3 > 5 4 < ou não
      - verdadeiro
  - 6 +max(1, 2, 3) + 4 * 5
    - 6 1 2 3 max + 4 5 * +
      - 29

Exemplos – Avaliação de Expressões

Algortimo para avaliar uma expressão Pós-Fixa genérica

- Para todos os elementos elem da Expressão
  - Se elem é um operador
    - Calcular o número de argumentos
    - Retirar os argumentos da pilha
    - Calcular o resultado
    - Introduzir o resultado na pilha
  - Senão
    - Introduz na elem pilha
- Retorna elemento que ficou na pilha

Exemplos – Avaliação de Expressões

Conversão de uma expressão In-Fixa em Pós-Fixa



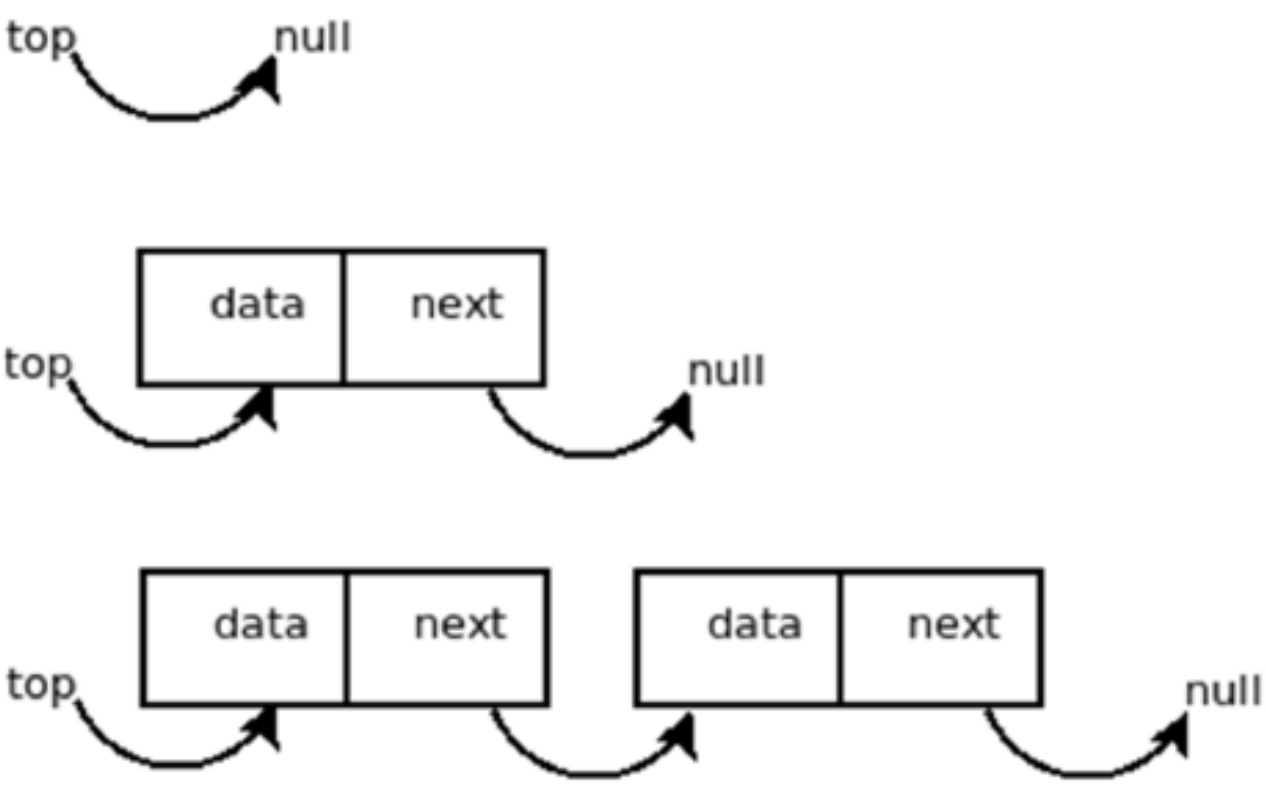
Exemplos – Avaliação de Expressões

- Inicializar String POSFIXA a nulo
- Para todos os elementos elem da string INFIXA
  - Se elem for um parentesis a abrir
    - Introduzir elem na pilha
  - Senão Se elem for um parentesis a fechar
    - Retirar todos os operadores até encontar o parentesis a abrir
  - Senão Se elem é um operador
    - X1 é o elemento disponível na pilha
    - Se a prioridade do operador elem for maior que x1
      - Introduzir elem na pilha
    - Senão
      - Retirar da pilha os elementos que tiverem prioridade maior ou igual a x1
      - Introduzir elem na pilha
  - Senão
    - Introduz na elem na string POSFIXA
- Retornar POSFIXA

Pilhas sem Tamanho Máximo

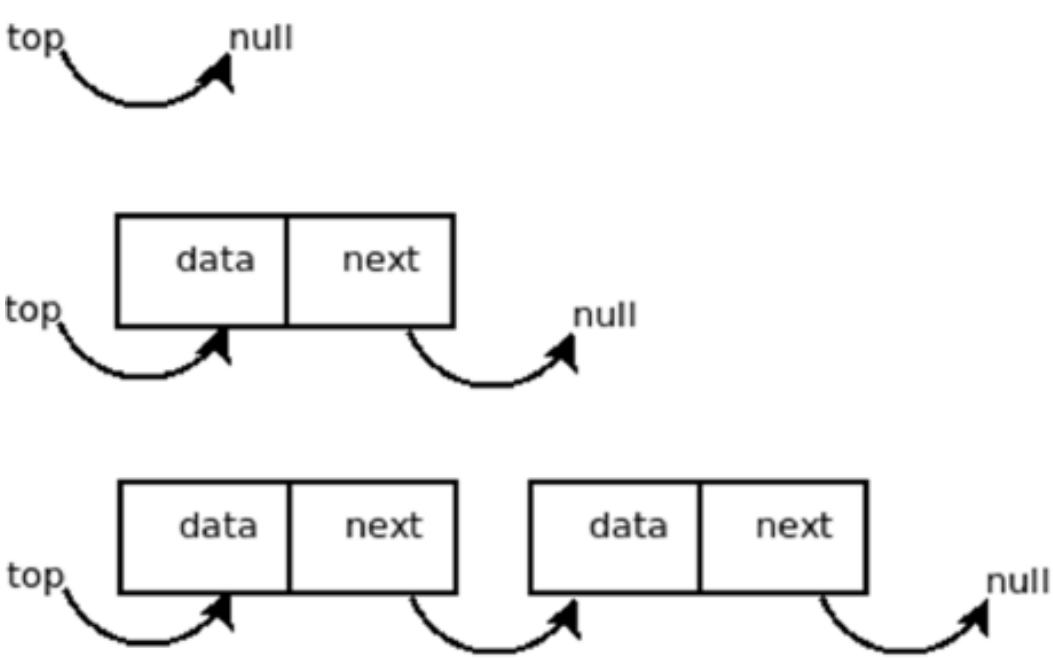
```
Item {
    Object data
    Item next
}

    Stack {
        Item top
    }
```



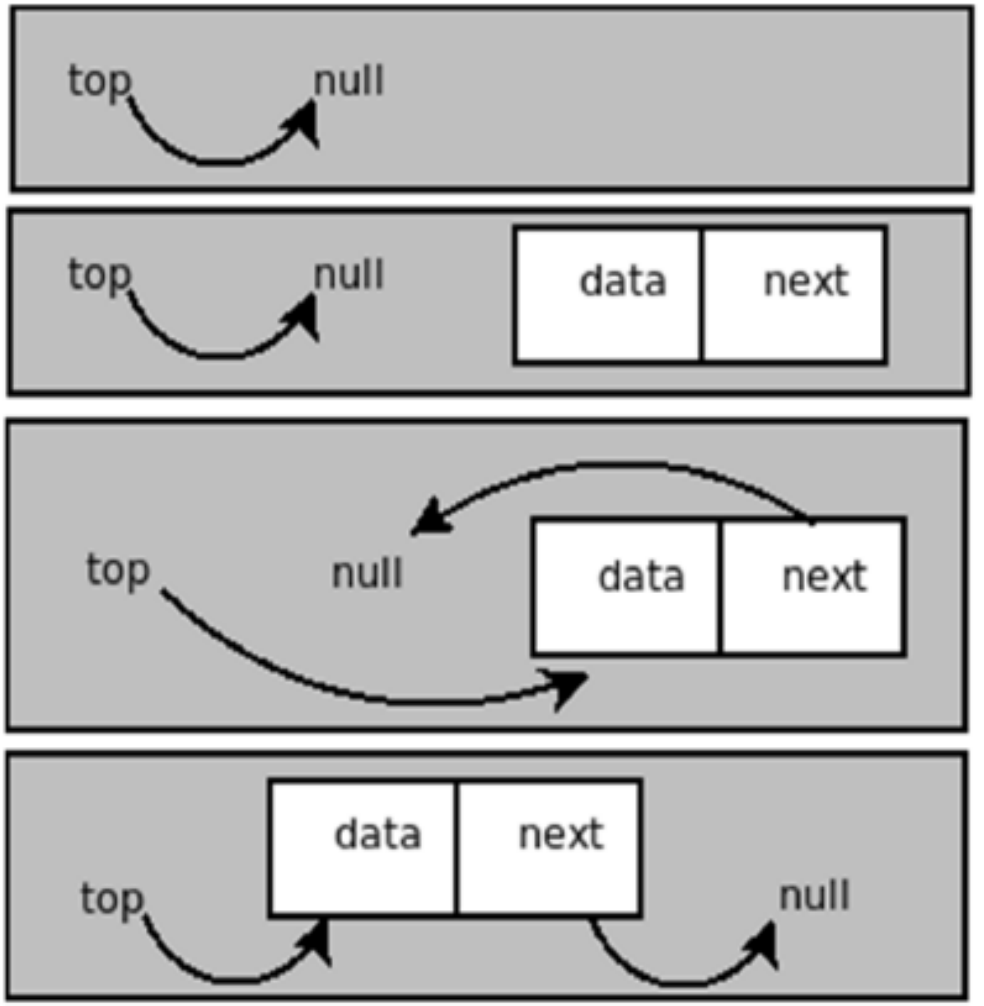
Empty

```
boolean empty() {
  Se topo = null
    devolve verdadeiro
  Senão
    devolve falso
}
```

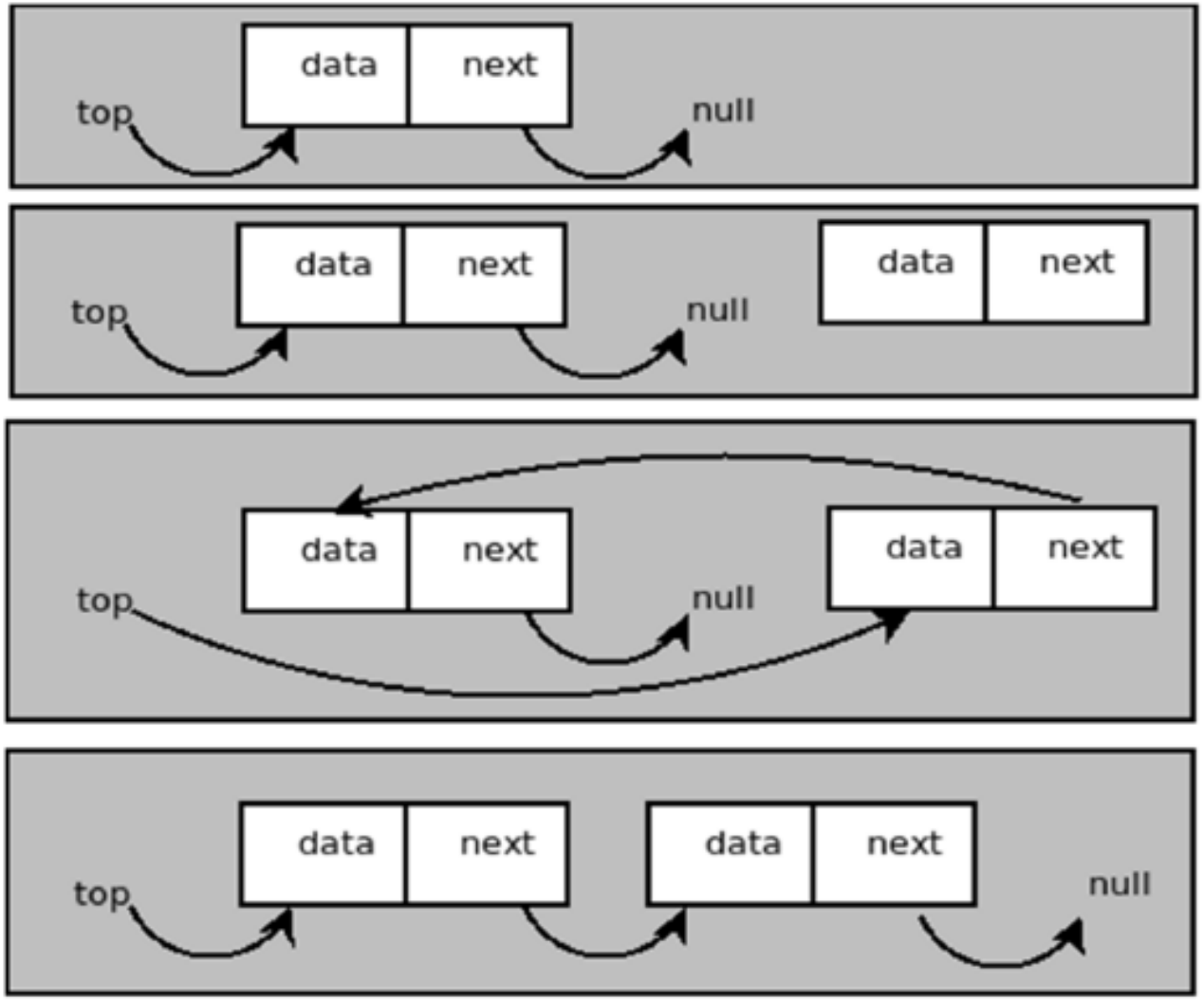


Push

```
void push(Object data) {
    novo(item)
    item.data = data
    item.next = top
    top = item
}
```



Push


data
next
top
null
data
next
top
null
data
next
data
next
top
null
data
next
data
next
top
null

Pop & Peek

```
Object pop() {
    data = top.data
    top = top.next
    devolve data
}
```

```
Object peek() {
    devolve top.data
}
```